

# BHTTP/1: a binary HTTP-like protocol over TCP

Version 1. Everything a stranger needs to write a compatible client or server. MUST, SHOULD and MAY are used in the RFC 2119 sense.

## 1. Model

BHTTP/1 runs over one persistent TCP connection; there is no assigned port. A client asks for a resource by path, the server answers with a status, headers and an opaque byte body. Exchanges are sequential and the connection stays open between them; a peer ends it by closing TCP, and no header controls that. TCP read and write boundaries mean nothing: receivers rebuild frames from the byte stream with exact reads, senders write every byte. A client never opens a second connection or reconnects silently to send another request. All multi-byte integers are unsigned and big-endian.

## 2. Frame

Every message is a frame: a 12-byte header followed by Length payload bytes, frames back to back.

| Offset | Size | Field     | Meaning                                                                            |
|--------|------|-----------|------------------------------------------------------------------------------------|
| 0      | 4    | Length    | payload bytes after the header (header not counted), 0 to 16384                    |
| 4      | 1    | Type      | 1 REQUEST, 2 RESPONSE, 3 DATA, 4 ERROR; any other value is unknown                 |
| 5      | 1    | Flags     | bit 0 (0x01) END_STREAM: last frame of this stream; other bits reserved, sent as 0 |
| 6      | 2    | Reserved  | sender sends 0, receiver ignores it                                                |
| 8      | 4    | Stream ID | non-zero on REQUEST, RESPONSE and DATA; ERROR uses 0                               |

Widths: Length is 32 bits although 14 would fit the 16 KiB cap, which keeps the header 4-byte aligned and lets a later version raise the cap without changing the layout. Type has 251 values left for extensions, Flags has one bit used and seven for later, and Reserved is room for a future field. Stream ID is a full 32 bits (HTTP/2 uses 31 bits, and 24 for Length) because  $2^{32}$  exchanges per connection is ample and the field stays a plain integer, with no bit to mask.

**Receiver algorithm** (client and server alike):

- Read exactly 12 bytes. EOF before the first byte is a clean close; EOF later inside a frame means truncation: discard the partial frame, send nothing, close.
- If Length > 16384: send ERROR(1) and close (if the oversized frame is itself an ERROR, just close). No memory is allocated from an unchecked Length.
- If Type is unknown: read and discard exactly Length bytes and carry on. Flags, Reserved and Stream ID of an unknown frame are not examined, and it never closes the connection.
- Read exactly Length bytes. If Type is 1, 2 or 3 and Stream ID is 0: send ERROR(2) and close. Otherwise handle the frame.

## 3. Payloads

| Frame    | Payload layout (the payload must end exactly after the last field)                                                                                                                   |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REQUEST  | Method u8 (1 = GET)   Path Length u16 (1 to 4096)   Path (UTF-8, explicit length, no terminator)   Header Count u8   headers. Flags: END_STREAM required (there is no request body). |
| RESPONSE | Status u16 (100 to 599)   Header Count u8   headers                                                                                                                                  |
| DATA     | Body bytes, unmodified: never decoded, normalized or terminated. At most 16384 per frame.                                                                                            |
| ERROR    | Code u16 (1 FRAME_TOO_LARGE, 2 INVALID_STREAM_ID, 3 PROTOCOL_ERROR)   Msg Length u16   Msg (UTF-8, for humans only, no sensitive detail). Connection-level: Stream ID 0.             |

**Headers.** The first byte of a header decides its form.

| Form                | Layout                                                                                     |
|---------------------|--------------------------------------------------------------------------------------------|
| Known (ID 1 to 255) | ID u8   Value Length u16   Value. The name comes from the static table; it is not sent.    |
| Custom (ID 0)       | 0   Name Length u16 (1 to 255)   Name (lowercase a-z 0-9 - _ .)   Value Length u16   Value |

Static table: 1 content-type, 2 content-length, 3 server, 4 date, 5 connection, 6 content-encoding, 7 cache-control, 8 last-modified.

- A table name is always sent by its ID; a custom header whose name equals a table name is malformed. Duplicates are allowed and keep their order.
- IDs 9 to 255 are unassigned: parse them with the known layout and ignore them. Values are opaque bytes; only content-length has a syntax (1 to 19 ASCII digits, compared numerically). IDs 4 to 8 are reserved names with no behavior; connection never controls the connection.

## 4. Exchange

- The client uses stream IDs 1, 2, 3, ... one per request, never reused or wrapped (after 4294967295 it sends nothing more). Every request consumes an ID, even one answered 400. The server accepts any non-zero ID and echoes it on every frame of its answer.

- Client: REQUEST with END\_STREAM. Server: RESPONSE, then zero or more DATA. END\_STREAM is on the RESPONSE when the body is empty, otherwise on the last DATA. There is never a trailing empty DATA frame, even when the size is an exact multiple of 16384. Receivers tolerate empty DATA frames. The body is the DATA payloads concatenated.
- Pipelined requests are answered in order and frames of different streams never interleave.

## 5. Errors, and what each fault causes

| Situation                                                                                                                                                                                                                                                                                                            | Outcome                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Well-framed REQUEST that is invalid: method not 1; a length that is zero, too large or runs past the payload; missing or trailing bytes; invalid UTF-8 (RFC 3629); bad header; END_STREAM missing or an undefined flag bit set; a path that breaks the rules in section 6. Also a RESPONSE or DATA sent to a server. | RESPONSE 400 on that stream, empty body, connection continues. Each bad frame gets its own 400. |
| Valid request, file missing, directory without index, or path resolves outside the root                                                                                                                                                                                                                              | RESPONSE 404, empty body                                                                        |
| Server fault before the RESPONSE is sent / after it (for example a file shorter than announced)                                                                                                                                                                                                                      | RESPONSE 500 / ERROR(3) and close, never a clean END_STREAM                                     |
| Length above 16384; Stream ID 0 on types 1 to 3                                                                                                                                                                                                                                                                      | ERROR(1) / ERROR(2) on stream 0, then close                                                     |
| ERROR received                                                                                                                                                                                                                                                                                                       | Close the connection; an ERROR is never answered                                                |

After sending an ERROR the sender shuts down its write side, reads and discards input for at most 1 s and 64 KiB, then closes, so a reset does not destroy the ERROR. A peer that just sent an ERROR treats a reset like EOF. Statuses a v1 server emits: 200, 400, 404, 500.

## 6. File serving (bserve <root> <port>)

- **Valid path** (else 400), the same on every platform: starts with /; no byte below 0x20 or 0x7F; none of \ : \* ? " < > |; no //; no . or .. segment; no segment ending in . or a space; no segment whose text before the first . is CON PRN AUX NUL COM1-9 LPT1-9 in any case. There is no percent-decoding and no query string: bytes are matched literally.
- **Mapping:** / is root/index.html; a directory, with or without a trailing slash, serves its index.html; a trailing slash on a regular file is 404. Symbolic links and junctions are resolved: inside the root they are served, outside they are 404.
- **Response headers**, in this order: 200 has content-type, content-length (the exact body size, taken from the open file), server. 400, 404 and 500 have content-length 0 and server. The body is the file's raw bytes.

## 7. Client rules

- Skip unknown frame types. After a request expect a RESPONSE on its own stream, then DATA on the same stream until END\_STREAM. Anything else is a failure: a frame on another stream, DATA before RESPONSE, a second RESPONSE, an ERROR, EOF or truncation before END\_STREAM, a status outside 100 to 599, a malformed RESPONSE payload (a length running past it, bytes left after the last header, a custom name length of 0 or over 255), or a content-length (every occurrence) that differs from the bytes received, which it may report as soon as the body exceeds it. After a failure the client closes the connection and never reuses it. Timeouts are the client's own policy.
- A client sends an ERROR only for the connection faults in section 2 and for a REQUEST arriving from a server (ERROR(3)). It ignores undefined flag bits on RESPONSE and DATA, while a server rejects them on REQUEST. It checks the structure of response headers but not their names: a custom name that would be invalid in a request is accepted and ignored, and never read as the table header of the same name. Status 400 or above is an error (a command-line client exits non-zero); 100 to 399 is not, and 3xx is not followed.

## 8. Limits and extensibility

Frame payload 16384 bytes; path 4096; headers per message 255; custom name 255; a header value is at most 65535 by its encoding and in practice bounded by the frame. A whole REQUEST or RESPONSE payload is one frame. Every length is checked against these limits and the bytes remaining before it is used. v1 has no version field. Later versions add new frame types (skipped by v1 peers), and assign flag bits and header IDs (ignored by v1 receivers).